

Characterizing and Bridging the Diagnostic Gap in eBPF Verifier Rejections

Yusheng Zheng^{*1,2}, Zhengjie Ji^{*3}, Weichen Tao^{*4}, Xiangyu Gao⁵, Jianchang Su⁶, Wei Zhang⁶, Andi Quinn¹, Dan Williams³

¹UC Santa Cruz ²eunomia-bpf ³Virginia Tech ⁴Telecom Paris ⁵University of Washington ⁶University of Connecticut

Abstract

eBPF lets developers run custom programs inside the Linux kernel, where a verifier proves each program safe. However, when the verifier rejects a program, the unclear error makes repair challenging: the error reports where verification stopped, not where the program lost the proof the verifier required. To quantify this gap, we conduct an empirical study of 235 reproduced rejections, showing that 47% of rejections return only EINVAL, one error string maps to as many as nine distinct root causes, and 10 of the 12 root causes are eBPF-specific. Repair thus requires both domain knowledge and locating where the proof was lost, yet existing tools only help developers read the error. We present `bpfix`, which reconstructs where the required proof was established and where it was lost from the verifier log, and prints a Rust-like diagnostic. To evaluate `bpfix` and the ability of LLMs to help repair, we construct a benchmark of 75 LLM repair tasks. Current models achieve 0–37% one-shot success with the raw log, and replacing the log with the `bpfix` localization improves repair by 11–21pp, suggesting that locating where the proof was lost is key to guiding repair. `bpfix` is available at <https://github.com/eunomia-bpf/bpfix>.

1 Introduction

eBPF lets developers run custom programs inside the Linux kernel for networking, security, and observability [1, 4]. To load a program, a developer writes it in C or Rust, compiles it to eBPF bytecode, and submits it to a verifier. The verifier proves the program safe by tracking abstract values along every execution path [5, 11].

However, the unclear verifier rejection makes repair challenging. Figure 1 illustrates this diagnostic gap: a packet pointer that the program bounds-checks is rejected at a later load with R5 `invalid mem access ‘scalar’`, on a line the developer has no reason to suspect. Without a way to locate where the proof was lost, developers must rely on repeated trial and error [3].

To quantify this gap, we conduct an empirical study of 235 rejections reproduced from Stack Overflow, GitHub issues, fix commits, and kernel selftests. We find that 191 are program bugs spanning 12 root causes, while the other 44 reject correct source due to compiler optimizations that obscure

pointer types, environment misconfiguration, or verifier limitations. Of the 12 root causes, 10 are eBPF-specific (e.g., packet bounds, `dynptr` lifetime), so repair requires domain knowledge beyond general programming. The terminal error provides little help: 47% of rejections return EINVAL, and one error string maps to as many as nine distinct root causes.

Existing tools do not close this gap. PrettyVerifier [14] and the BPF Verifier Visualizer [8] help developers read the error with regular expressions or visualizations, but neither locates where the proof was lost. Error Explainers in other domains like Rust walk a structure the checker builds, such as an AST or typing constraints [13, 22], but the eBPF verifier exposes no such artifact.

To improve the diagnostic process, we introduce `bpfix`, which reconstructs where the required proof was established and where it was lost from the verifier log, and prints a Rust-like diagnostic (Figure 1).

To measure the ability of LLMs to fix verifier rejections, we construct `bpfix-bench`, a benchmark of 75 repair tasks. With the raw verifier log, current LLMs achieve only 0–37% one-shot repair across three models ranging from 3B to 27B parameters, showing that verifier rejections remain difficult even for capable models. To see whether better diagnostics help automated repair, we replace the log with the `bpfix` localization, which improves repair by 11–21pp, with the gain largest at the stages where a repair must restore the verifier-visible proof. The result suggests that locating where the proof was lost, rather than only where verification stopped, is key to guiding repair.

This paper makes the following contributions:

- An empirical study of 235 reproduced rejections showing the terminal error reports where verification stopped, leaving the developer to find where the proof was lost (§2).
- `bpfix`, a technique that reconstructs where the required proof was established and lost from the verifier log (§3).
- `bpfix-bench`, a benchmark of 75 repair tasks for evaluating LLM-based verifier repair (§5).

2 An Empirical Study of Verifier Rejections

A rejection confronts the developer with a low-level log, and this section studies what that log records about real rejections and where it falls short. We describe how a

^{*}Equal contribution.

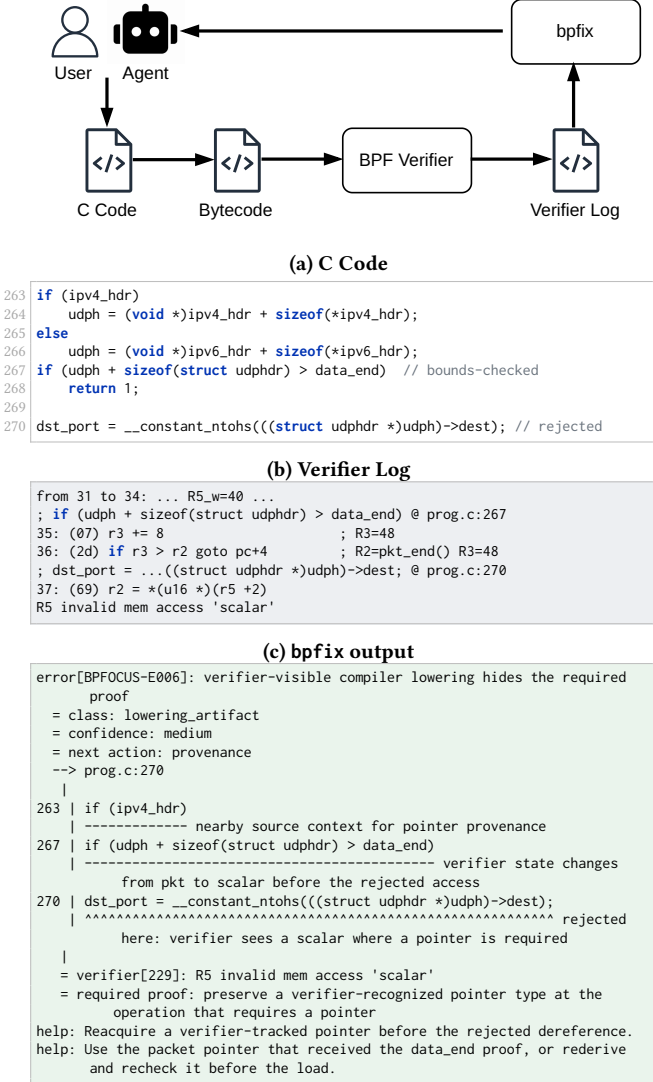


Figure 1. bpfix on a real rejection (Stack Overflow 53136145). The verifier returns one register-level line; from the same log, bpfix reconstructs the discarded proof and localizes the rejection to the point where the required proof was lost, mapping each span back to source.

developer reads a rejection (§2.1), and then we assemble bpfix-empirical, a dataset of real rejections (§2.2), characterize the rejections in it (§2.3), and measure the log’s diagnostic gap (§2.4).

2.1 Background: The Verifier Rejection

Before the kernel runs an eBPF program, the verifier must prove the program safe. The verifier proves safety one instruction at a time, building for each value the proof that it is used safely, such as a packet pointer staying within bounds, and exploring every path. A program that fails a check is rejected, and the verifier returns a log of the abstract state

```

1 // correct source; -O0 lowers the field read into a scalar
2 static int add_one(int x) { return x + 1; }
3 int prog(struct xdp_md *ctx) { return add_one(ctx->rx_queue_index) & 1; }

verifier: R2 invalid mem access 'scalar'

```

Figure 2. A compiler-lowering rejection: correct source that the verifier reports as invalid mem access ‘scalar’.

after each instruction, ending in a one-line terminal error at the rejected instruction.

The terminal error names the rejection location, the operation where verification stopped, which the verifier reaches only after the program lost the proof at an earlier instruction. To repair the program so it passes the verifier, the developer must learn which proof the verifier required and where the program failed to keep it, because a fix has to restore that proof. The error names only the operation that needed the proof, so the developer reconstructs the required proof and its loss by hand from the trace.

2.2 Reproducing Real Rejections in bpfix-empirical

To study rejections that developers actually hit, we assemble bpfix-empirical, a dataset of real verifier rejections reproduced under one fixed toolchain. The dataset draws on 936 candidate reports from four sources: Stack Overflow questions, GitHub issues, GitHub fix commits, and kernel selftests. We rebuild each candidate and load it into kernel 6.15.11 with clang 18 at log level 2, and 235 of the 936 reject under this build and form bpfix-empirical. The rest are dropped because they do not reject under our toolchain, need a specific environment, or lack the source to rebuild.

Each bpfix-empirical case carries the faulty source and the developer’s own fix, both from its report. The source and the fix give the ground truth for the rejection and where its repair landed, which the next two subsections use.

2.3 Characterizing the Rejections

We characterize each bpfix-empirical rejection by where its repair lands. In 191 of the 235 rejections the repair changes the program’s source. The other 44 reject correct source and are repaired elsewhere: in the compiler (18), the environment (14), or the verifier (12). For example, a context-field read that is provably safe is rejected because `-O0` lowers it into a scalar. The repair is a compiler flag that leaves the source untouched (Figure 2). The rest of this subsection examines the 191 program bugs.

A program bug fails a safety check, but the failed check does not name the source mistake. We therefore label each bug with its root cause, the source-level error the developer made. The 191 bugs fall into 12 root causes (Table 1), most specific to eBPF, such as proving a packet bound or pairing a dynptr with its lifetime.

Table 1. The 191 developer bugs fall into 12 root-cause categories. A category is the specific source-level error the developer made; the *Cases* column gives how many bugs each accounts for.

Root-cause category	Cases
Unclamped scalar used as offset or length	24
Corrupted or stale dynptr object	23
Packet access without a bound on every path	22
Missing null check	19
Pointer type or provenance mismatch	16
Unverified address dereferenced	16
Index exceeds object capacity	15
Context or contract misuse	15
Unpaired resource reference	15
Interrupt flag not restored in order	11
Probe signature mismatched with the ABI	9
Oversized or uninitialized stack buffer	6
Total	191

Table 2. The four most frequent message templates, with the cases sharing each template and the distinct root-cause categories they span. A template normalizes registers and offsets to placeholders.

Terminal message template	Cases	Categories
R# invalid mem access ‘scalar’	28	9
invalid access to packet	26	5
invalid access to map value	18	4
R# !read_ok	13	4

Takeaway #1: Most rejections (81%) are program bugs, but 19% reject correct source due to compiler, environment, or verifier issues. Of 12 root causes, 10 are eBPF-specific, so repair requires domain knowledge.

2.4 Measuring the Log’s Localization Gap

With the root causes labeled, we find the terminal error too coarse to name which one a rejection hit. The `errno` it carries does little to narrow the cause, as `EINVAL` alone tags 47% of all rejections. The error string is just as coarse, because one string maps to many root causes. To measure how often, we mask the registers and offsets in each string and group the 235 rejections by the normalized template. This collapses 167 distinct strings into 82 templates. Fifteen of the 82 templates each map to more than one root cause. The most frequent alone covers nine root causes across 28 rejections (Table 2).

Because one error matches many root causes, the error alone cannot identify which proof the program lost, or where, the two facts a repair needs. The trace, however, holds more than the terminal error. Under `log_level=2` the verifier

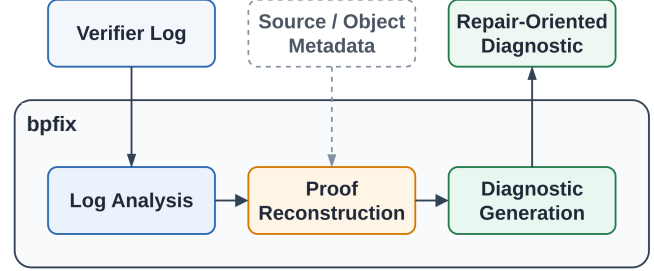


Figure 3. Overview of the `bpf ix` workflow. `bpf ix` analyzes verifier logs, reconstructs the missing proof, and generates a repair-oriented diagnostic.

prints its abstract state after every instruction, so the lost proof survives in the printed states, and §3 reconstructs the proof from those states to localize the loss.

Takeaway #2: The terminal error is too coarse to guide repair: 47% of rejections return `EINVAL`, and one error string maps to as many as nine root causes.

3 bpf ix Design and Implementation

The goal of `bpf ix` is to answer two questions the terminal error leaves open: which proof the verifier required, and where the program lost it. `bpf ix` reconstructs the proof lifecycle from the existing verifier log by parsing the per-instruction abstract states and tracking when evidence for the required proof appears, persists, and is lost. Figure 3 summarizes the three-stage workflow: log analysis normalizes evidence around the rejection, proof reconstruction identifies and tracks the required proof, and diagnostic generation turns the resulting evidence into guidance the developer can act on.

3.1 Architecture and Data Flow

`bpf ix` is implemented in approximately 23k lines of Rust as a pipeline whose only required input is an existing verifier log; optional source or object metadata improves instruction-to-source mapping. The Log Analysis stage extracts the terminal error and the per-instruction abstract states into a normalized evidence stream. The Proof Reconstruction stage maps the terminal error to a *proof family* (such as pointer provenance, packet bounds, or scalar range), identifies the required proof, and tracks evidence for that proof. The Diagnostic Generation stage selects spans and derives guidance for re-establishing the proof.

The result is a Rust-style plain-text diagnostic with a stable error identifier, the required proof, relevant spans and evidence, a loss point when observable, and `help:` guidance.

3.2 Reconstructing the Verifier-Visible Proof

bpfix first identifies which proof the verifier required. The terminal error identifies the rejected operation and immediate symptom, while nearby abstract state supplies parameters such as the affected register, access offset and size, scalar range, pointer type, or reference identifier. Together they identify the required proof at the *rejection location*, the instruction where the verifier stopped. For example, in Figure 1, R5 invalid mem access ‘scalar’ maps to a pointer-provenance proof. From the rejected load and nearby state, bpfix infers that R5 must still carry verifier-recognized packet-pointer provenance when the rejected load reads it.

bpfix then tracks evidence for that proof through the log: when the evidence appears (establishment), when it becomes incompatible (loss), and when the verifier requires it (rejection). When both establishment and loss are observed, the transition between them becomes the *loss point*. In the running example, an earlier state represents R5 as pkt(off=34,r=42), whereas a later state before instruction 37 represents it as scalar value 40. The final load dereferences R5, so bpfix reports the observed pointer-to-scalar transition as the loss point and instruction 37 as the rejection location.

3.3 Evidence-Aware Diagnostic Synthesis

Diagnostic synthesis adapts its guidance to the reconstructed proof lifecycle. bpfix localizes the evidence needed for repair: the rejected operation, the required proof at that operation, the loss point when the log exposes one, and the span and next action that guide the developer in re-establishing the proof. When the proof was never established, bpfix recommends introducing it, for example through a bounds check, null check, or reference acquisition. When the proof is visible and later lost, it recommends preserving or re-deriving it near the rejected use. Developers use the resulting guidance to choose and validate a repair through the normal compile-and-load workflow.

4 Evaluation

This section evaluates the bpfix diagnostic with two case studies on bpfix-empirical, the 235 reproduced rejections of §2.2. The first shows how the diagnostic recovers the repair information the terminal error omits, and the second shows how it separates the responsible layer behind a shared terminal error.

4.1 Recovering the Missing Repair Information

We examine one rejection from the aya project to show how bpfix recovers the repair information the terminal error omits.

As shown in Figure 4, the program treats the map object &globals as a map-value pointer, casting its address to __u64 * and reading and writing through the result. The

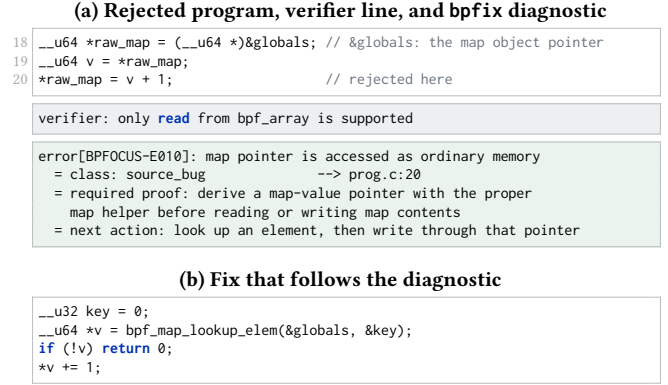


Figure 4. A real rejection from the aya project (issue 1002): (a) the rejected program, the verifier’s terminal line, and the bpfix diagnostic; (b) the developer’s fix.

verifier rejects the write with the terminal line only **read** from `bpf_array` is supported. The line is correct, but it gives only the symptom: the verifier sees a write through the map object where a map value is required.

From the same log, bpfix reconstructs the lost proof. The diagnostic states the proof the rejected write needs, a map-value pointer derived through a map helper before any access to map contents. It then localizes the loss to the cast that produced a raw map-object pointer. As shown in Figure 4, the developer’s fix follows that proof: it calls `bpf_map_lookup_elem`, checks the returned pointer, and updates the value through the result. The terminal line gives only the rejected operation, while the diagnostic recovers the lost proof and the point where the program lost it.

4.2 Separating the Responsible Layer

A second diagnostic gap is that one terminal verifier message can call for repairs in different layers. bpfix reports the responsible layer as a best-effort attribution, labeling a rejection `source_bug` for the source layer of §2 or `lowering_artifact` for the compiler layer. We inspect whether bpfix separates two rejections that share one terminal error but belong to different layers.

Figure 5 shows two rejections that both end with invalid mem access ‘scalar’. In Figure 5(a), the program constructs a packet pointer by casting an integer offset, `(void *) (sizeof(*eth) + nh_off)`. The resulting value has no verifier-tracked packet base, so the later packet-field load uses a scalar where the verifier requires a real pointer. bpfix reports `source_bug`: the source never establishes the pointer provenance the rejected load requires.

In Figure 5(b), the source expresses the intended packet-bound proof, deriving `udph` from an IPv4 or IPv6 header and bounds-checking it against `data_end` before the read. The rejection arises after compiler lowering merges the branch and hides the verifier-recognized packet-pointer

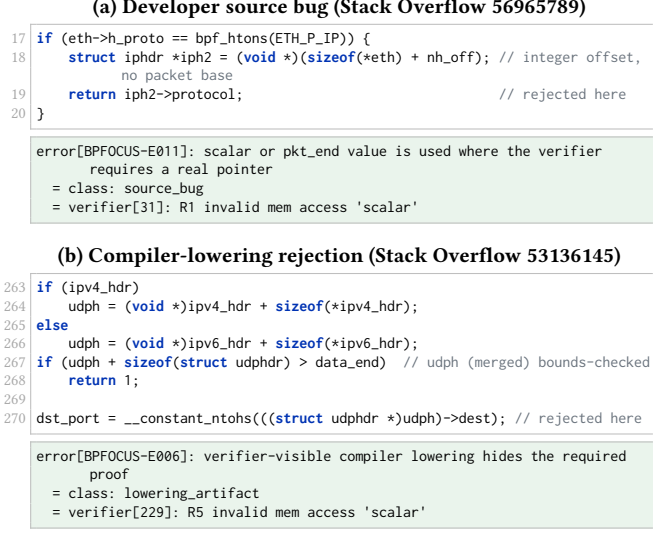


Figure 5. Two rejections that share the verifier message invalid mem access ‘scalar’ but need fixes in different layers: a source bug (a) and a compiler-lowering artifact (b). Each diagnostic is abridged to its error id, class, and the verifier line it explains.

type before the load. The upstream fix steers the compiler and leaves the source logic unchanged, so the layer label holds independently of bpf_{ix}. bpf_{ix} reports the case as lowering_artifact. On these two rejections, bpf_{ix} reports a different layer for each, separating a source bug from a lowering artifact behind one terminal error.

5 bpf_{ix}-bench: A Benchmark for LLM Verifier Repair

To measure the ability of LLMs to fix verifier rejections and evaluate whether bpf_{ix} helps, we construct bpf_{ix}-bench, a benchmark of 75 source-level repair tasks.

5.1 Benchmark Construction

bpf_{ix}-bench contains 75 repair tasks: 40 built around a required verifier proof and 35 minimized from open-source projects such as Cilium [1], xdp-tools [17], and bpftime [25]. Each task ships the buggy program with an executable test suite independent of bpf_{ix}, which accepts a fix only if it loads through the kernel verifier and passes the case’s functional and source-semantics checks. Because the tests are independent of bpf_{ix}, a fix earns credit only by passing the same checks a correct program must pass.

We score each task in two modes. The one-shot mode tests the diagnostic on its own, and the retry mode allows one failure-informed retry to test whether the help persists. Our primary model is Qwen3.6 27B, and we also run the hosted GLM 5.2 and the smaller Qwen2.5 3B at temperature zero. We include the small 3B model to see whether the gain survives

at low model capacity. For each task, a model receives the buggy program with either the raw verifier log or the bpf_{ix} diagnostic and writes a candidate fix for the test suite to judge.

5.2 Results

Replacing the raw verifier log with the bpf_{ix} diagnostic raises repair for every model, as shown in Figure 6. On Qwen3.6 27B, one-shot repair rises from 22 of 75 cases with the raw log to 38 with the diagnostic, and one retry raises the two counts to 30 and 44. GLM 5.2 shows the same pattern, from 28 to 38 in one shot and 52 against 47 with the retry. The small Qwen2.5 3B repairs none of the 75 from the raw log, and 8 from the diagnostic in one shot and 10 with one retry. The gain holds across all three models, which points to the diagnostic as its source.

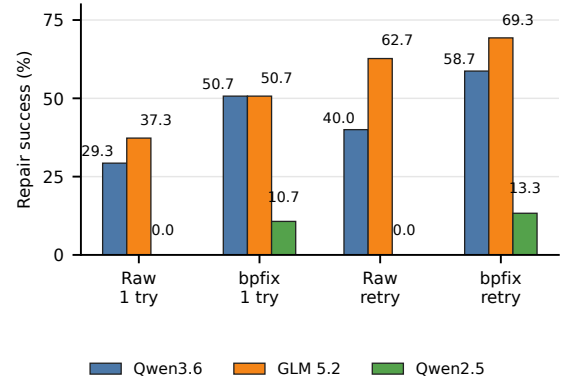


Figure 6. bpf_{ix}-bench repair success across three models. Bars are grouped by prompt mode and colored by model; retry bars use one failure-informed retry.

To locate the gain, we charge each failed one-shot candidate to the first stage it fails: returning a program, compiling, loading through the verifier, passing the functional test, or passing the source-semantics test. As Table 3 shows, the diagnostic mainly reduces verifier-load and source-semantics failures. On Qwen3.6 27B, load failures fall from 19 to 10 and source-semantics failures from 22 to 16, and GLM 5.2 shows the same two reductions, from 10 to 5 and from 25 to 22, with compile failures staying low for both models.

For the small Qwen2.5 3B, the raw log is often too weak a repair signal: 62 one-shot candidates still fail at verifier load, and three raw-log prompts exceed the model’s context window and return no program. With the shorter bpf_{ix} diagnostic, model-call failures disappear and verifier-load failures fall to 39, though compile failures rise from 7 to 14. bpf_{ix} gives even a small model enough verifier-facing information to attempt more meaningful repairs, while ordinary code-generation errors remain.

Table 3. Where repairs fail on `bpfix`-bench, by stage, for the one-attempt runs. The `bpfix` diagnostic mainly removes verifier-load (Load) and proof failures; compile and model-call (Call) failures stay low. Each entry counts failing cases at a stage, and pass count plus the row total is 75.

Model	Input	Compile	Load	Func.	Proof	Call
Qwen3.6 27B	raw	3	19	9	22	0
	<code>bpfix</code>	1	10	10	16	0
GLM 5.2	raw	1	10	11	25	0
	<code>bpfix</code>	1	5	9	22	0
Qwen2.5 3B	raw	7	62	0	3	3
	<code>bpfix</code>	14	39	6	8	0

Overall, the quantitative results match the two case studies. The diagnostic helps most at the stages where a repair must restore the verifier-visible proof, loading through the verifier and preserving the program’s source semantics. These stages are exactly where `bpfix` localizes the lost proof, so the model receives what a repair must restore.

6 Related Work

eBPF verification and safety. Prior eBPF tooling decides whether a program is safe or works around the verifier, leaving rejection diagnosis to the developer [3]. Offline reprovers such as PREVAIL establish safety over their own abstract domain [5], verifier audits check the soundness of its operators [16, 18, 19], and other work replaces the verifier with safe Rust [6], synthesizes passing programs [24], or isolates programs at runtime [9, 10, 12, 23]. None replays the rejection log to reconstruct where the required proof was lost.

Error diagnosis and fault localization. Tools that diagnose rejections outside eBPF read facts the checker built: type-error diagnosis reasons over typing constraints [13, 15, 22], spectrum-based fault localization needs many test runs [7, 21], program slicing needs the dependence graph [20], and unsat-core extraction needs a solver’s search artifacts [2]. For eBPF, Pretty Verifier uses regular expressions to map the terminal error to a source line and hint [14], and BPF Verifier Visualizer provides a frontend that renders per-instruction state interactively [8], but neither localizes where the proof was lost. `bpfix` reconstructs the proof lifecycle from the verifier’s own printed state, recovering register types, scalar ranges, and pointer provenance [11].

7 Conclusion

When the verifier rejects an eBPF program, its terminal error marks where verification stopped, which can lie far from where the program lost the required proof. Reading only the rejection log, `bpfix` reconstructs the required proof’s

lifecycle from the verifier’s per-instruction states and localizes where the proof was lost. On `bpfix`-bench, the `bpfix` localization raises a downstream model’s one-shot repair from 22 to 38 of the 75 cases, with every accepted fix passing the kernel verifier independently of `bpfix`.

References

- [1] Cilium Authors. 2026. Cilium: eBPF-based Networking, Observability, and Security. <https://github.com/cilium/cilium>. Open-source project, accessed 2026.
- [2] A. Cimatti, A. Griggio, and R. Sebastiani. 2011. Computing Small Unsatisfiable Cores in Satisfiability Modulo Theories. *Journal of Artificial Intelligence Research* 40 (April 2011), 701–728. doi:10.1613/jair.3196
- [3] Mugdha Deokar, Jingyang Men, Lucas Castanheira, Ayush Bhardwaj, and Theophilus A. Benson. 2024. An Empirical Study on the Challenges of eBPF Application Development. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions* (Sydney, NSW, Australia) (eBPF ’24). Association for Computing Machinery, New York, NY, USA, 1–8. doi:10.1145/3672197.3673429
- [4] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, Toke Høiland-Jørgensen, Simone Ferlin-Reiter, Simo Sorce, and Anna Brunström. 2024. The eBPF Runtime in the Linux Kernel. arXiv:2410.00026 [cs.OS] <https://arxiv.org/abs/2410.00026>
- [5] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzkzy, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 1069–1084. doi:10.1145/3314221.3314590
- [6] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the language-verifier gap with safe and usable kernel extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association, Boston, MA, 325–342. <https://www.usenix.org/conference/atc25/presentation/jia>
- [7] James A. Jones and Mary Jean Harrold. 2005. Empirical evaluation of the tarantula automatic fault-localization technique. In *Proceedings of the 20th IEEE/ACM International Conference on Automated Software Engineering* (Long Beach, CA, USA) (ASE ’05). Association for Computing Machinery, New York, NY, USA, 273–282. doi:10.1145/1101908.1101949
- [8] libbpf. 2025. bpfv: BPF Verifier Visualizer. <https://github.com/libbpf/bpfv>. Accessed 2026-06-25.
- [9] Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. 2023. Unleashing Unprivileged eBPF Potential with Dynamic Sandboxing. In *Proceedings of the 1st Workshop on EBPF and Kernel Extensions* (New York, NY, USA) (eBPF ’23). Association for Computing Machinery, New York, NY, USA, 42–48. doi:10.1145/3609021.3609301
- [10] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. 2024. SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions. In *Proceedings of the 2024 on Cloud Computing Security Workshop* (Salt Lake City, UT, USA) (CCSW ’24). Association for Computing Machinery, New York, NY, USA, 80–94. doi:10.1145/3689938.3694781
- [11] Linux kernel documentation. 2025. eBPF verifier. <https://docs.kernel.org/bpf/verifier.html>. The Linux Kernel documentation, Documentation/bpf/verifier.rst.
- [12] Hongyi Lu, Shuai Wang, Yechang Wu, Wanning He, and Fengwei Zhang. 2024. MOAT: Towards Safe BPF Kernel Extension. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1153–1170. <https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi>
- [13] Zvonimir Pavlinovic, Tim King, and Thomas Wies. 2014. Finding minimum type error sources. *SIGPLAN Not.* 49, 10 (Oct. 2014), 525–542.

- doi:10.1145/2714064.2660230
- [14] Rosario Rizza, Riccardo Sisto, and Fulvio Valenza. 2025. Design and implementation of a tool to improve error reporting for eBPF code. In *2025 IEEE International Conference on Cyber Security and Resilience (CSR)* (Chania, Crete, Greece). IEEE, Piscataway, NJ, USA, 214–219. doi:10.1109/CSR64739.2025.11130075
 - [15] Eric L. Seidel, Huma Sibghat, Kamalika Chaudhuri, Westley Weimer, and Ranjit Jhala. 2017. Learning to blame: localizing novice type errors with data-driven diagnosis. *Proc. ACM Program. Lang.* 1, OOPSLA, Article 60 (Oct. 2017), 27 pages. doi:10.1145/3138818
 - [16] Hao Sun and Zhendong Su. 2024. Validating the eBPF verifier via state embedding. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) (*OSDI'24*). USENIX Association, USA, Article 33, 14 pages.
 - [17] The XDP Project. 2026. xdp-tools: Utilities and Example Programs for Use with XDP. <https://github.com/xdp-project/xdp-tools>. Open-source project, accessed 2026.
 - [18] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2022. Sound, precise, and fast abstract interpretation with tristate numbers. In *Proceedings of the 20th IEEE/ACM International Symposium on Code Generation and Optimization* (Virtual Event, Republic of Korea) (*CGO '22*). IEEE Press, Piscataway, NJ, USA, 254–265. doi:10.1109/CGO53902.2022.9741267
 - [19] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 226–251. doi:10.1007/978-3-031-37709-9_12
 - [20] Mark Weiser. 1981. Program slicing. In *Proceedings of the 5th International Conference on Software Engineering* (San Diego, California, USA) (*ICSE '81*). IEEE Press, Piscataway, NJ, USA, 439–449.
 - [21] W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 08 (Aug. 2016), 707–740. doi:10.1109/TSE.2016.2521368
 - [22] Danfeng Zhang and Andrew C. Myers. 2014. Toward general diagnosis of static errors. *SIGPLAN Not.* 49, 1 (Jan. 2014), 569–581. doi:10.1145/2578855.2535870
 - [23] Peihua Zhang, Chenggang Wu, Xiangyu Meng, Yinqian Zhang, Mingfan Peng, Shiyang Zhang, Bing Hu, Mengyao Xie, Yuanming Lai, Yan Kang, and Zhe Wang. 2024. HIVE: A Hardware-assisted Isolated Execution Environment for eBPF on AArch64. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 163–180. <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-peihua>
 - [24] Yusheng Zheng, Yiwei Yang, Maolin Chen, and Andrew Quinn. 2024. Kgent: Kernel Extensions Large Language Model Agent. In *Proceedings of the ACM SIGCOMM 2024 Workshop on EBPF and Kernel Extensions* (Sydney, NSW, Australia) (*eBPF '24*). Association for Computing Machinery, New York, NY, USA, 30–36. doi:10.1145/3672197.3673434
 - [25] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending applications safely and efficiently. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation* (Boston, MA, USA) (*OSDI '25*). USENIX Association, Boston, MA, USA, 557–574. <https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng>